# BLUEPRINT: A2 DocIntel — Production RAG with RAGAS Eval, Hybrid Search & Drift Monitoring

> ***Authoritative architecture document.*** *Codex must not deviate from decisions in this document. All architectural decisions are final. Implementation questions reference this document first.* `DECISION NEEDED` *markers require user input before that section can be executed.* `ENV REQUIRED` *markers require a runtime credential to be available before that phase begins.*
>
> ***Blueprint Author:*** *Claude Code (* `/flagship-plan` *)* ***Date:*** *2026-04-13* ***Status:*** *Draft*

---

## 0. Project Identity

| Field | Value |
|---|---|
| Project Code | A2 |
| Project Name | DocIntel — Production RAG with RAGAS Eval, Hybrid Search & Drift Monitoring |
| Repo Path | `d:\Mehul-Projects\DocIntel- RAG with RAGAS Eval, Hybrid Search & Drift Monitoring` |
| GitHub | https://github.com/Mehulupase01/DocIntel--RAG-with-RAGAS-Eval-Hybrid-Search-Drift-Monitoring |
| Branch | main |
| Codex Charter | `D:\Mehul-Projects\Codex_Project_Execution_Charter.md` |
| Claude Charter | `D:\Mehul-Projects\Claude_Planning_Charter.md` |

### One-Line Description

Production-grade document intelligence service that answers regulatory questions over the EU AI Act PDF using hybrid retrieval (BM25 + pgvector + cross-encoder reranking), citation-grounded generation, automated RAGAS quality scoring in CI, LangSmith request tracing, and Evidently drift monitoring — exposed as a FastAPI service with a Streamlit ops dashboard.

### Problem Statement

Most "RAG demos" are vector-only chatbots that silently degrade in production: retrieval drifts as the corpus grows, faithfulness regresses unnoticed across model swaps, and there is no eval harness gating PRs. DocIntel solves this by making retrieval, generation, and evaluation first-class: every retrieval is hybrid + reranked, every answer is citation-grounded, every PR is gated by RAGAS thresholds in CI, every request is traced in LangSmith, and weekly drift reports flag retrieval quality regressions. The system targets the EU AI Act corpus (a high-stakes regulatory document) so accuracy and provenance are non-negotiable.

### Success Criteria

- [ ] RAGAS faithfulness >= 0.89 mean on the 100-case golden fixture
- [ ] RAGAS context precision >= 0.92 mean on the 100-case golden fixture
- [ ] RAGAS context recall >= 0.85 mean on the 100-case golden fixture

- [ ] Hybrid + reranked retrieval beats vector-only by >= 15% on context precision
- [ ] p95 end-to-end `/api/v1/answer` latency <= 4.0s on local CPU (excluding cold model load)
- [ ] All phases pass their verification commands
- [ ] `docker compose up -d` brings up API + Postgres/pgvector + dashboard end-to-end
- [ ] GitHub Actions workflow runs RAGAS eval on every PR and fails on threshold regression
- [ ] Evidently weekly drift job emits a JSON + HTML report stored in DB and on disk
- [ ] README accurately describes verified behavior with reproducible commands

---

# 1. System Architecture

## Architecture Pattern

**Chosen Pattern:** Modular Monolith (single FastAPI service, internally partitioned by domain module) with a co-located Streamlit dashboard process and a Postgres+pgvector data plane.

**Justification:** The system has one deployable hot path (ingest → retrieve → generate → log) with a small number of asynchronous side-jobs (eval runs, drift reports). Microservices add network and ops cost without any independent scaling need. A modular monolith keeps each domain ( `ingestion` , `retrieval` , `generation` , `evaluation` , `monitoring` ) cleanly testable while sharing one Postgres schema and one deployment unit. The Streamlit dashboard runs as a separate process for read-only ops visibility; it talks to the same Postgres directly for analytic reads and to the API for actions.

**Primary Alternative Rejected:** Separate microservices per domain — rejected because every cross-domain call (retrieval → generation → eval logging) currently happens inside one request lifecycle and would become a network hop with no scaling justification at this corpus size (low-thousands of chunks).

## System Diagram

```
┌──────────────────────────────────────────────────────────────┐
│                        Clients                               │
│   curl / Postman      Streamlit Dashboard      GitHub Actions  │
└──────────────┬──────────────────┬──────────────────┬──────────┘
               │ HTTPS            │ HTTPS            │ HTTPS
               ▼                  ▼                  ▼
┌──────────────────────────────────────────────────────────────┐
│                 FastAPI Service (apps/api)                    │
│   /documents   /search   /answer   /eval   /drift   /health   │
└──────┬──────────┬──────────┬──────────┬──────────┬──────────┘
   │           │           │           │           │
   ▼           ▼           ▼           ▼           ▼
┌──────┐  ┌─────────┐  ┌──────────┐  ┌────────┐  ┌────────┐
│Ingest│  │Retrieval│  │Generation│  │Eval    │  │Drift   │
│      │  │ BM25 +  │  │ LLM call │  │ RAGAS  │  │Evidently│
│PDF→  │  │ pgvector│  │ +citation│  │ harness│  │ weekly  │
│chunks│  │ + RRF + │  │ extractor│  │ +CI gate│ │ report  │
│+embed│  │ reranker│  │          │  │        │  │        │
└──────┘  └─────────┘  └──────────┘  └────────┘  └────────┘
   │           │           │           │           │
   └───────────┴───────────┴───────────┴───────────┘
                           │
                           ▼
```

```
        ┌──────────────────────────┐
        │   PostgreSQL 16 + pgvector │
        │  documents, chunks (vector │
        │  + tsvector), queries,     │
        │  retrievals, answers,      │
        │  citations, eval_runs,     │
        │  eval_cases, drift_reports │
        └──────────────┬───────────┘
                       │
        ┌──────────────▼──────────────┐
        │  Local model cache          │
        │  bge-small-en-v1.5          │
        │  ms-marco-MiniLM-L-6-v2     │
        └──────────────────────────┘


External (optional, env-gated):
  • OpenRouter (LLM for /answer + RAGAS judge)
  • LangSmith (request tracing)
```

### Component Responsibilities

| Component | Responsibility | Technology |
|---|---|---|
| API Layer | HTTP routing, auth, request/response validation, OpenAPI | FastAPI 0.135.2 |
| Ingestion Module | PDF parsing, semantic chunking, embedding, indexing | pypdf 5.4.0, sentence-transformers 3.4.1 |
| Retrieval Module | BM25 (tsvector), vector ANN (pgvector), RRF fusion, cross-encoder rerank | pgvector 0.4.1, sentence-transformers 3.4.1 |
| Generation Module | Prompt assembly, LLM call, citation extraction | httpx 0.28.1, OpenRouter |
| Evaluation Module | RAGAS metric computation, golden fixture runner, CI gate | ragas 0.3.5, langchain-openai 0.3.13 |
| Observability Module | LangSmith tracing, Prometheus metrics, structlog | langsmith 0.3.20, prometheus-client 0.23.1, structlog 25.4.0 |
| Drift Module | Evidently report generation, scheduled job, persistence | evidently 0.6.7, APScheduler 3.11.0 |
| ORM / DB | Persistence, async sessions, migrations | SQLAlchemy 2.0.48 + Alembic 1.18.4 |
| Dashboard | Read-only ops UI: eval trends, drift, cost, traces | Streamlit 1.45.0 |
| Container Runtime | Local + CI orchestration | Docker 27.x, Docker Compose 2.x |

---

## 2. Technology Stack

All versions are pinned. Do not use "latest". Do not substitute without user approval. Cutoff verification: see Risk R1.

### Backend (apps/api)

| Package | Pinned Version | Purpose | Justification |
|---|---|---|---|
| Python | 3.12.x | Runtime | Matches sibling projects (CrewAI, GraphRAG); broad library support |
| fastapi | 0.135.2 | API framework | Same version as sibling CrewAI api; mature async stack |
| uvicorn[standard] | 0.42.0 | ASGI server | Standard FastAPI pairing |
| sqlalchemy | 2.0.48 | Async ORM | 2.x async API; matches sibling pattern |
| alembic | 1.18.4 | DB migrations | Standard SQLAlchemy companion |
| asyncpg | 0.31.0 | Postgres async driver | Required for SQLAlchemy async on Postgres |
| aiosqlite | 0.21.0 | SQLite async driver | Test-only DB |
| pgvector | 0.4.1 | Vector type for SQLAlchemy + Postgres | Native pgvector ORM bindings |
| pydantic | 2.11.9 | Validation | FastAPI native, V2 perf |
| pydantic-settings | 2.10.1 | Typed env config | `.env` loading with type checks |
| python-multipart | 0.0.20 | File upload parsing | Required for `POST /documents` PDF upload |
| httpx | 0.28.1 | Outbound HTTP (LLM + tests) | Async, used by AsyncClient in tests |
| pypdf | 5.4.0 | PDF text + structure extraction | Pure-Python, no system deps |
| sentence-transformers | 3.4.1 | Embeddings + cross-encoder | Pinned model: BAAI/bge-small-en-v1.5 + ms-marco-MiniLM-L-6-v2 |
| torch | 2.6.0 | sentence-transformers backend | CPU wheel sufficient for inference |
| numpy | 1.26.4 | Vector ops | sentence-transformers dep, pinned to avoid 2.x churn |
| langchain | 0.3.27 | RAGAS LLM/embeddings adapter | Required transitively by RAGAS; not used in production answer path |
| langchain-openai | 0.3.13 | OpenAI-compatible LLM adapter for RAGAS | Used by RAGAS judge to call OpenRouter |
| langsmith | 0.3.20 | Distributed tracing | Optional; env-gated |

| | | | |
|---|---|---|---|
| ragas | 0.3.5 | RAG evaluation metrics | faithfulness, context_precision, context_recall, answer_relevancy |
| evidently | 0.6.7 | Drift detection + reports | Embedding + retrieval drift |
| apscheduler | 3.11.0 | In-process weekly drift job | Avoids extra worker dep; suitable for single-instance |
| prometheus-client | 0.23.1 | Metrics | `/metrics` endpoint |
| structlog | 25.4.0 | Structured logging | JSON logs in prod, key-value in dev |
| pyjwt | 2.10.1 | API key signing (future-proof) | Pinned now to avoid later churn |
| python-dotenv | 1.1.0 | `.env` support | pydantic-settings dep |
| pytest | 9.0.2 | Test runner | Matches sibling |
| pytest-asyncio | 1.0.0 | Async tests | `asyncio_mode = "auto"` |
| pytest-cov | 6.0.0 | Coverage | CI gate |
| ruff | 0.15.7 | Lint + format | Matches sibling |

### Dashboard (apps/dashboard)

| Package | Pinned Version | Purpose |
|---|---|---|
| Python | 3.12.x | Runtime (shares image base) |
| streamlit | 1.45.0 | Ops dashboard |
| pandas | 2.2.3 | Tabular display + aggregations |
| plotly | 5.24.1 | Charts (eval trends, drift series) |
| httpx | 0.28.1 | Calls into API |
| sqlalchemy | 2.0.48 | Read-only Postgres queries |
| psycopg[binary] | 3.2.4 | Sync Postgres driver for Streamlit |

### Infrastructure

| Tool | Version | Purpose |
|---|---|---|
| Docker | 27.x | Containerization |
| Docker Compose | 2.x | Local + CI orchestration |
| PostgreSQL | 16.4 (image: `pgvector/pgvector:pg16`) | DB + vector index |
| Node.js | 20.x LTS | Required only for `npx md-to-pdf` blueprint export |

### AI/ML

| Library / Model | Pinned Version | Purpose |
|---|---|---|
| BAAI/bge-small-en-v1.5 | revision pinned via env `EMBEDDING_MODEL_REVISION` | Document + query embedding (384-dim) |
| cross-encoder/ms-marco-MiniLM-L-6-v2 | revision pinned via env `RERANKER_MODEL_REVISION` | Cross-encoder reranker |
| OpenRouter (LLM provider) | API version v1 | Production answer generation + RAGAS judge |
| Default generation model | `anthropic/claude-haiku-4-5` | Fast, cheap, citation-friendly. Any OpenRouter-hosted model ID is accepted via `AnswerRequest.model` and `DEFAULT_GENERATION_MODEL`; the LLM client is OpenRouter-native (`OPENROUTER_BASE_URL`), so switching providers is a one-env-var change. |
| Default judge model | `openai/gpt-4o-mini` | Cheap RAGAS judge with stable scoring. Any OpenRouter-hosted model ID is accepted via `EvalRunCreate.judge_model` and `DEFAULT_JUDGE_MODEL`; `langchain-openai` is configured with `base_url=OPENROUTER_BASE_URL` so RAGAS routes through OpenRouter end-to-end. |

### Rejected Alternatives

| Alternative | Considered For | Rejection Reason |
|---|---|---|
| Qdrant / Weaviate | Vector store | pgvector keeps everything in one DB; no extra service to operate |
| `rank_bm25` (Python in-memory) | BM25 backend | Doesn't scale with corpus growth; Postgres `tsvector + GIN` is production-grade |
| LangChain end-to-end pipeline | Full RAG framework | Hides the seams that this project is explicitly built to expose; we use LangChain only as the RAGAS adapter |
| Celery + Redis | Background jobs | One in-process APScheduler weekly job is sufficient; Redis adds no value here |
| Next.js dashboard | UI | Streamlit ships in hours, target audience is internal ops, not public users |
| ColBERT late-interaction | Reranking | Heavier infra (PLAID index); cross-encoder rerank on top-50 hits the precision target at lower cost |
| OpenAI direct | LLM provider | OpenRouter unifies provider switching for the model-comparison story (Phase 5) |

---

## 3. Repository Structure

Codex must create exactly this structure. Do not add directories not listed here without justification.

```
DocIntel- RAG with RAGAS Eval, Hybrid Search & Drift Monitoring/
├── apps/
│   ├── api/
│   │   ├── src/
│   │   │   └── docintel/
│   │   │       ├── __init__.py
│   │   │       ├── main.py                      # FastAPI app factory, lifespan, CORS,
routers
│   │   │       ├── config.py                    # pydantic-settings Settings
│   │   │       ├── database.py                  # Async engine, session factory, get_db
│   │   │       ├── logging_setup.py             # structlog config
│   │   │       ├── auth.py                      # API key dependency
│   │   │       ├── models/
│   │   │       │   ├── __init__.py
│   │   │       │   ├── base.py                  # DeclarativeBase
│   │   │       │   ├── document.py
│   │   │       │   ├── chunk.py
│   │   │       │   ├── query.py
│   │   │       │   ├── retrieval.py
│   │   │       │   ├── answer.py
│   │   │       │   ├── citation.py
│   │   │       │   ├── eval_run.py
│   │   │       │   ├── eval_case.py
│   │   │       │   └── drift_report.py
│   │   │       ├── schemas/
│   │   │       │   ├── __init__.py
│   │   │       │   ├── document.py
│   │   │       │   ├── search.py
│   │   │       │   ├── answer.py
│   │   │       │   ├── eval.py
│   │   │       │   ├── drift.py
│   │   │       │   └── common.py                # error envelope, pagination
│   │   │       ├── routers/
│   │   │       │   ├── __init__.py
│   │   │       │   ├── health.py
│   │   │       │   ├── documents.py
│   │   │       │   ├── search.py
│   │   │       │   ├── answer.py
│   │   │       │   ├── eval.py
│   │   │       │   ├── drift.py
│   │   │       │   └── metrics.py
│   │   │       ├── services/
│   │   │       │   ├── __init__.py
│   │   │       │   ├── ingestion/
│   │   │       │   │   ├── __init__.py
│   │   │       │   │   ├── pdf_loader.py
│   │   │       │   │   ├── chunker.py           # semantic / heading-aware
│   │   │       │   │   ├── embedder.py          # bge-small-en-v1.5
│   │   │       │   │   └── pipeline.py          # orchestrates load→chunk→embed→persist
│   │   │       │   ├── retrieval/
│   │   │       │   │   ├── __init__.py
```

```
│   │   │       │   │   ├── bm25.py               # tsvector queries
│   │   │       │   │   ├── vector.py             # pgvector ANN
│   │   │       │   │   ├── fusion.py             # Reciprocal Rank Fusion
│   │   │       │   │   ├── reranker.py           # cross-encoder
│   │   │       │   │   └── hybrid.py             # public façade
│   │   │       │   ├── generation/
│   │   │       │   │   ├── __init__.py
│   │   │       │   │   ├── prompt.py             # citation-grounded template
│   │   │       │   │   ├── llm_client.py         # OpenRouter HTTP client
│   │   │       │   │   ├── citation_extractor.py # parses [doc:X#chunk:Y] markers
│   │   │       │   │   └── answerer.py           # public façade
│   │   │       │   ├── evaluation/
│   │   │       │   │   ├── __init__.py
│   │   │       │   │   ├── ragas_runner.py
│   │   │       │   │   ├── fixture_loader.py
│   │   │       │   │   ├── thresholds.py
│   │   │       │   │   └── ci_gate.py            # CLI entry for GitHub Actions
│   │   │       │   ├── monitoring/
│   │   │       │   │   ├── __init__.py
│   │   │       │   │   ├── langsmith_setup.py
│   │   │       │   │   ├── metrics.py            # prometheus collectors
│   │   │       │   │   └── tracing.py            # request middleware
│   │   │       │   └── drift/
│   │   │       │       ├── __init__.py
│   │   │       │       ├── evidently_runner.py
│   │   │       │       ├── scheduler.py          # APScheduler weekly job
│   │   │       │       └── reporter.py
│   │   │       └── tools/
│   │   │           ├── __init__.py
│   │   │           ├── ingest_eu_ai_act.py       # CLI: download + ingest
│   │   │           ├── run_eval.py               # CLI: full RAGAS run
│   │   │           ├── run_drift.py              # CLI: one-shot drift
│   │   │           └── benchmark_retrieval.py    # vector-only vs hybrid vs reranked
│   │   ├── alembic/
│   │   │   ├── env.py
│   │   │   ├── script.py.mako
│   │   │   └── versions/
│   │   │       ├── 001_foundation.py
│   │   │       ├── 002_documents_and_chunks.py
│   │   │       ├── 003_queries_retrievals_answers.py
│   │   │       ├── 004_eval_runs_and_cases.py
│   │   │       └── 005_drift_reports.py
│   │   ├── alembic.ini
│   │   ├── tests/
│   │   │   ├── __init__.py
│   │   │   ├── conftest.py
│   │   │   ├── test_health.py
│   │   │   ├── test_documents.py
│   │   │   ├── test_chunker.py
│   │   │   ├── test_embedder.py
│   │   │   ├── test_bm25.py
│   │   │   ├── test_vector.py
```

```
│   │   │   ├── test_fusion.py
│   │   │   ├── test_reranker.py
│   │   │   ├── test_search_endpoint.py
│   │   │   ├── test_answer_endpoint.py
│   │   │   ├── test_citation_extractor.py
│   │   │   ├── test_eval_runner.py
│   │   │   ├── test_drift_runner.py
│   │   │   └── fixtures/
│   │   │       ├── tiny_pdf.pdf                 # 3-page synthetic PDF
│   │   │       └── synthetic_qa.json            # 5-case test fixture
│   │   ├── pyproject.toml
│   │   └── Dockerfile
│   └── dashboard/
│       ├── app.py                               # Streamlit entrypoint
│       ├── pages/
│       │   ├── 1_Eval_Trends.py
│       │   ├── 2_Drift_Reports.py
│       │   ├── 3_Cost_and_Latency.py
│       │   └── 4_Retrieval_Explorer.py
│       ├── lib/
│       │   ├── __init__.py
│       │   ├── db.py                            # read-only SQLAlchemy session
│       │   └── api_client.py
│       ├── pyproject.toml
│       └── Dockerfile
├── fixtures/
│   ├── eu_ai_act_qa_v1.json                    # 100 manually curated Q&A pairs
│   ├── eu_ai_act_qa_v1.schema.json             # JSON schema for fixture
│   └── README.md                                # provenance + curation method
├── docs/
│   ├── HANDOFF.md
│   ├── PROGRESS.md
│   ├── DECISIONS.md
│   ├── architecture.md
│   ├── data_model.md
│   ├── api.md
│   ├── ingestion.md
│   ├── retrieval.md
│   ├── evaluation.md
│   ├── drift.md
│   ├── deployment.md
│   └── verification.md
├── ops/
│   ├── docker/
│   │   ├── compose.full.yml                     # api + dashboard + postgres + (optional)
langsmith proxy
│   │   └── postgres-init/
│   │       └── 001_pgvector.sql                 # CREATE EXTENSION vector
│   └── github-actions/
│       └── ragas-eval.yml                       # source for .github/workflows/ragas-
eval.yml
├── .github/
```

```
│   └── workflows/
│       ├── ci.yml                          # lint + unit tests on every PR
│       └── ragas-eval.yml                  # RAGAS regression gate on every PR
├── data/
│   └── source/
│       └── README.md                       # how to fetch the EU AI Act PDF
├── BLUEPRINT.md                            # this file
├── CLAUDE.md                               # working memory (Phase 1)
├── README.md                               # final phase
├── docker-compose.yml                      # dev compose (api + db only)
├── .env.example
├── .gitignore
├── .gitattributes                          # already present
├── LICENSE                                 # already present
└── pyproject.toml                          # workspace root (ruff config + tooling)
```

---

## 4. Data Models

### Entity Overview

| Entity | Table | PK Type | Description |
|---|---|---|---|
| Document | `documents` | UUID | A source PDF (e.g., EU AI Act) |
| Chunk | `chunks` | UUID | A retrievable unit of a document with embedding + tsvector |
| Query | `queries` | UUID | A user search/answer request (one row per request) |
| Retrieval | `retrievals` | UUID | One retrieved chunk for a query, with all score components |
| Answer | `answers` | UUID | LLM-generated answer for a query |
| Citation | `citations` | UUID | A chunk cited by an answer with span text |
| EvalRun | `eval_runs` | UUID | One full RAGAS run over a fixture |
| EvalCase | `eval_cases` | UUID | One fixture case scored within a run |
| DriftReport | `drift_reports` | UUID | One Evidently drift snapshot |

### Detailed SQLAlchemy Models

Implement exactly these schemas. Do not add or remove fields without user approval. All models inherit from `Base` defined in `apps/api/src/docintel/models/base.py` .

```python
# apps/api/src/docintel/models/base.py
from sqlalchemy.orm import DeclarativeBase

class Base(DeclarativeBase):
    pass
```

```python
# apps/api/src/docintel/models/document.py
import uuid
from datetime import datetime
from enum import Enum
from typing import List, Optional
from sqlalchemy import String, Text, Integer, DateTime, Enum as SAEnum, Index
from sqlalchemy.dialects.postgresql import JSONB, UUID
from sqlalchemy.orm import Mapped, mapped_column, relationship
from sqlalchemy.sql import func
from .base import Base

class DocumentStatus(str, Enum):
    PENDING = "pending"
    INGESTING = "ingesting"
    READY = "ready"
    FAILED = "failed"

class Document(Base):
    __tablename__ = "documents"

    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid.uuid4)
    title: Mapped[str] = mapped_column(String(512), nullable=False)
    source_uri: Mapped[Optional[str]] = mapped_column(String(2048), nullable=True)
    sha256: Mapped[str] = mapped_column(String(64), nullable=False, unique=True)
    page_count: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    byte_size: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    status: Mapped[DocumentStatus] = mapped_column(
        SAEnum(DocumentStatus, name="document_status"), nullable=False,
default=DocumentStatus.PENDING
    )
    error_message: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    metadata_json: Mapped[dict] = mapped_column(JSONB, nullable=False, default=dict)
    ingested_at: Mapped[Optional[datetime]] = mapped_column(DateTime(timezone=True),
nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
server_default=func.now())
    updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
server_default=func.now(), onupdate=func.now())

    chunks: Mapped[List["Chunk"]] = relationship(back_populates="document", cascade="all,
delete-orphan")

    __table_args__ = (
        Index("ix_documents_status", "status"),
        Index("ix_documents_sha256", "sha256", unique=True),
    )
```

```python
# apps/api/src/docintel/models/chunk.py
import uuid
```

```python
from datetime import datetime
from typing import Optional
from sqlalchemy import String, Text, Integer, DateTime, ForeignKey, Index, Computed
from sqlalchemy.dialects.postgresql import JSONB, UUID, TSVECTOR
from sqlalchemy.orm import Mapped, mapped_column, relationship
from sqlalchemy.sql import func
from pgvector.sqlalchemy import Vector
from .base import Base

EMBEDDING_DIM = 384  # bge-small-en-v1.5

class Chunk(Base):
    __tablename__ = "chunks"

    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid.uuid4)
    document_id: Mapped[uuid.UUID] = mapped_column(
        UUID(as_uuid=True), ForeignKey("documents.id", ondelete="CASCADE"), nullable=False
    )
    ordinal: Mapped[int] = mapped_column(Integer, nullable=False)  # 0..N-1 within doc
    text: Mapped[str] = mapped_column(Text, nullable=False)
    token_count: Mapped[int] = mapped_column(Integer, nullable=False)
    char_start: Mapped[int] = mapped_column(Integer, nullable=False)
    char_end: Mapped[int] = mapped_column(Integer, nullable=False)
    page_start: Mapped[int] = mapped_column(Integer, nullable=False)
    page_end: Mapped[int] = mapped_column(Integer, nullable=False)
    section_path: Mapped[Optional[str]] = mapped_column(String(1024), nullable=True)  # e.g.
"Title III > Chapter 2 > Article 9"
    embedding: Mapped[list[float]] = mapped_column(Vector(EMBEDDING_DIM), nullable=False)
    tsv: Mapped[str] = mapped_column(
        TSVECTOR,
        Computed("to_tsvector('english', text)", persisted=True),
        nullable=False,
    )
    metadata_json: Mapped[dict] = mapped_column(JSONB, nullable=False, default=dict)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
server_default=func.now())

    document: Mapped["Document"] = relationship(back_populates="chunks")

    __table_args__ = (
        Index("ix_chunks_document_id", "document_id"),
        Index("ix_chunks_doc_ord", "document_id", "ordinal", unique=True),
        Index("ix_chunks_tsv", "tsv", postgresql_using="gin"),
        Index(
            "ix_chunks_embedding_hnsw", "embedding",
            postgresql_using="hnsw",
            postgresql_with={"m": 16, "ef_construction": 64},
            postgresql_ops={"embedding": "vector_cosine_ops"},
        ),
    )
```

```python
# apps/api/src/docintel/models/query.py
import uuid
from datetime import datetime
from enum import Enum
from typing import Optional, List
from sqlalchemy import String, Text, Integer, Float, DateTime, Enum as SAEnum
from sqlalchemy.dialects.postgresql import JSONB, UUID
from sqlalchemy.orm import Mapped, mapped_column, relationship
from sqlalchemy.sql import func
from .base import Base

class RetrievalStrategy(str, Enum):
    VECTOR_ONLY = "vector_only"
    BM25_ONLY = "bm25_only"
    HYBRID = "hybrid"
    HYBRID_RERANKED = "hybrid_reranked"

class Query(Base):
    __tablename__ = "queries"

    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid.uuid4)
    query_text: Mapped[str] = mapped_column(Text, nullable=False)
    strategy: Mapped[RetrievalStrategy] = mapped_column(
        SAEnum(RetrievalStrategy, name="retrieval_strategy"), nullable=False
    )
    top_k: Mapped[int] = mapped_column(Integer, nullable=False)
    rerank_top_n: Mapped[Optional[int]] = mapped_column(Integer, nullable=True)
    alpha: Mapped[Optional[float]] = mapped_column(Float, nullable=True)  # for non-RRF
fusion variants
    rrf_k: Mapped[Optional[int]] = mapped_column(Integer, nullable=True, default=60)
    session_id: Mapped[Optional[str]] = mapped_column(String(64), nullable=True)
    latency_ms: Mapped[Optional[int]] = mapped_column(Integer, nullable=True)
    metadata_json: Mapped[dict] = mapped_column(JSONB, nullable=False, default=dict)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
server_default=func.now())

    retrievals: Mapped[List["Retrieval"]] = relationship(back_populates="query",
cascade="all, delete-orphan")
    answers: Mapped[List["Answer"]] = relationship(back_populates="query", cascade="all,
delete-orphan")
```

```python
# apps/api/src/docintel/models/retrieval.py
import uuid
from datetime import datetime
from typing import Optional
from sqlalchemy import Integer, Float, DateTime, ForeignKey, Index
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import Mapped, mapped_column, relationship
from sqlalchemy.sql import func
```

```python
from .base import Base

class Retrieval(Base):
    __tablename__ = "retrievals"

    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid.uuid4)
    query_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), ForeignKey("queries.id",
ondelete="CASCADE"), nullable=False)
    chunk_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), ForeignKey("chunks.id",
ondelete="CASCADE"), nullable=False)
    rank: Mapped[int] = mapped_column(Integer, nullable=False)
    bm25_score: Mapped[Optional[float]] = mapped_column(Float, nullable=True)
    vector_score: Mapped[Optional[float]] = mapped_column(Float, nullable=True)
    fused_score: Mapped[Optional[float]] = mapped_column(Float, nullable=True)
    rerank_score: Mapped[Optional[float]] = mapped_column(Float, nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
server_default=func.now())

    query: Mapped["Query"] = relationship(back_populates="retrievals")

    __table_args__ = (
        Index("ix_retrievals_query_id_rank", "query_id", "rank"),
    )
```

```python
# apps/api/src/docintel/models/answer.py
import uuid
from datetime import datetime
from typing import Optional, List
from sqlalchemy import String, Text, Integer, Float, DateTime, ForeignKey, Index
from sqlalchemy.dialects.postgresql import JSONB, UUID
from sqlalchemy.orm import Mapped, mapped_column, relationship
from sqlalchemy.sql import func
from .base import Base

class Answer(Base):
    __tablename__ = "answers"

    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid.uuid4)
    query_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), ForeignKey("queries.id",
ondelete="CASCADE"), nullable=False)
    model: Mapped[str] = mapped_column(String(128), nullable=False)
    prompt_text: Mapped[str] = mapped_column(Text, nullable=False)
    response_text: Mapped[str] = mapped_column(Text, nullable=False)
    prompt_tokens: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    completion_tokens: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    cost_usd: Mapped[float] = mapped_column(Float, nullable=False, default=0.0)
    latency_ms: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    finish_reason: Mapped[Optional[str]] = mapped_column(String(64), nullable=True)
    metadata_json: Mapped[dict] = mapped_column(JSONB, nullable=False, default=dict)
```

```
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
server_default=func.now())

    query: Mapped["Query"] = relationship(back_populates="answers")
    citations: Mapped[List["Citation"]] = relationship(back_populates="answer",
cascade="all, delete-orphan")

    __table_args__ = (
        Index("ix_answers_query_id", "query_id"),
        Index("ix_answers_created_at", "created_at"),
    )
```

```
# apps/api/src/docintel/models/citation.py
import uuid
from sqlalchemy import Text, Integer, ForeignKey, Index
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import Mapped, mapped_column, relationship
from .base import Base

class Citation(Base):
    __tablename__ = "citations"

    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid.uuid4)
    answer_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
ForeignKey("answers.id", ondelete="CASCADE"), nullable=False)
    chunk_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), ForeignKey("chunks.id",
ondelete="CASCADE"), nullable=False)
    ordinal: Mapped[int] = mapped_column(Integer, nullable=False)  # citation order in the
answer text
    span_text: Mapped[str] = mapped_column(Text, nullable=False)   # excerpt the LLM cited

    answer: Mapped["Answer"] = relationship(back_populates="citations")

    __table_args__ = (
        Index("ix_citations_answer_id_ord", "answer_id", "ordinal"),
    )
```

```
# apps/api/src/docintel/models/eval_run.py
import uuid
from datetime import datetime
from enum import Enum
from typing import Optional, List
from sqlalchemy import String, Integer, Float, DateTime, Enum as SAEnum, Boolean
from sqlalchemy.dialects.postgresql import JSONB, UUID
from sqlalchemy.orm import Mapped, mapped_column, relationship
from sqlalchemy.sql import func
from .base import Base

class EvalRunStatus(str, Enum):
```

```
    RUNNING = "running"
    PASSED = "passed"
    FAILED = "failed"
    ERRORED = "errored"

class EvalRun(Base):
    __tablename__ = "eval_runs"

    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid.uuid4)
    suite_version: Mapped[str] = mapped_column(String(32), nullable=False)
    git_sha: Mapped[Optional[str]] = mapped_column(String(40), nullable=True)
    generation_model: Mapped[str] = mapped_column(String(128), nullable=False)
    judge_model: Mapped[str] = mapped_column(String(128), nullable=False)
    retrieval_strategy: Mapped[str] = mapped_column(String(64), nullable=False)
    status: Mapped[EvalRunStatus] = mapped_column(SAEnum(EvalRunStatus,
name="eval_run_status"), nullable=False, default=EvalRunStatus.RUNNING)
    total_cases: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    cases_passed: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    faithfulness_mean: Mapped[Optional[float]] = mapped_column(Float, nullable=True)
    context_precision_mean: Mapped[Optional[float]] = mapped_column(Float, nullable=True)
    context_recall_mean: Mapped[Optional[float]] = mapped_column(Float, nullable=True)
    answer_relevancy_mean: Mapped[Optional[float]] = mapped_column(Float, nullable=True)
    thresholds_json: Mapped[dict] = mapped_column(JSONB, nullable=False, default=dict)
    started_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
server_default=func.now())
    finished_at: Mapped[Optional[datetime]] = mapped_column(DateTime(timezone=True),
nullable=True)

    cases: Mapped[List["EvalCase"]] = relationship(back_populates="run", cascade="all,
delete-orphan")
```

```
# apps/api/src/docintel/models/eval_case.py
import uuid
from datetime import datetime
from typing import Optional, List
from sqlalchemy import String, Text, Float, DateTime, ForeignKey, Boolean, Index
from sqlalchemy.dialects.postgresql import JSONB, UUID
from sqlalchemy.orm import Mapped, mapped_column, relationship
from sqlalchemy.sql import func
from .base import Base

class EvalCase(Base):
    __tablename__ = "eval_cases"

    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid.uuid4)
    run_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), ForeignKey("eval_runs.id",
ondelete="CASCADE"), nullable=False)
    fixture_case_id: Mapped[str] = mapped_column(String(64), nullable=False)
    question: Mapped[str] = mapped_column(Text, nullable=False)
```

```
    ground_truth: Mapped[str] = mapped_column(Text, nullable=False)
    generated_answer: Mapped[str] = mapped_column(Text, nullable=False)
    contexts_json: Mapped[List[str]] = mapped_column(JSONB, nullable=False, default=list)
    faithfulness: Mapped[Optional[float]] = mapped_column(Float, nullable=True)
    context_precision: Mapped[Optional[float]] = mapped_column(Float, nullable=True)
    context_recall: Mapped[Optional[float]] = mapped_column(Float, nullable=True)
    answer_relevancy: Mapped[Optional[float]] = mapped_column(Float, nullable=True)
    passed: Mapped[bool] = mapped_column(Boolean, nullable=False, default=False)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
server_default=func.now())

    run: Mapped["EvalRun"] = relationship(back_populates="cases")

    __table_args__ = (
        Index("ix_eval_cases_run_id", "run_id"),
    )
```

```
# apps/api/src/docintel/models/drift_report.py
import uuid
from datetime import datetime
from enum import Enum
from typing import Optional
from sqlalchemy import String, Float, DateTime, Enum as SAEnum
from sqlalchemy.dialects.postgresql import JSONB, UUID
from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy.sql import func
from .base import Base

class DriftStatus(str, Enum):
    OK = "ok"
    WARNING = "warning"
    ALERT = "alert"

class DriftReport(Base):
    __tablename__ = "drift_reports"

    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid.uuid4)
    window_start: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)
    window_end: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)
    reference_window_start: Mapped[datetime] = mapped_column(DateTime(timezone=True),
nullable=False)
    reference_window_end: Mapped[datetime] = mapped_column(DateTime(timezone=True),
nullable=False)
    embedding_drift_score: Mapped[Optional[float]] = mapped_column(Float, nullable=True)
    query_drift_score: Mapped[Optional[float]] = mapped_column(Float, nullable=True)
    retrieval_quality_delta: Mapped[Optional[float]] = mapped_column(Float, nullable=True)
    status: Mapped[DriftStatus] = mapped_column(SAEnum(DriftStatus, name="drift_status"),
nullable=False, default=DriftStatus.OK)
    html_path: Mapped[Optional[str]] = mapped_column(String(1024), nullable=True)
    payload_json: Mapped[dict] = mapped_column(JSONB, nullable=False, default=dict)
```

```
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
server_default=func.now())
```

### Pydantic Schemas (key ones)

```
# apps/api/src/docintel/schemas/common.py
from typing import Generic, TypeVar, List, Optional, Any
from pydantic import BaseModel

T = TypeVar("T")

class ErrorBody(BaseModel):
    code: str
    message: str
    detail: dict = {}

class ErrorEnvelope(BaseModel):
    error: ErrorBody

class PageMeta(BaseModel):
    page: int
    per_page: int
    total: int

class Paginated(BaseModel, Generic[T]):
    data: List[T]
    meta: PageMeta
```

```
# apps/api/src/docintel/schemas/search.py
from typing import List, Optional, Literal
import uuid
from pydantic import BaseModel, Field

class SearchRequest(BaseModel):
    query: str = Field(min_length=1, max_length=2000)
    top_k: int = Field(default=10, ge=1, le=50)
    strategy: Literal["vector_only", "bm25_only", "hybrid", "hybrid_reranked"] =
"hybrid_reranked"
    rerank_top_n: int = Field(default=50, ge=10, le=200)
    rrf_k: int = Field(default=60, ge=1, le=1000)
    document_ids: Optional[List[uuid.UUID]] = None

class RetrievedChunk(BaseModel):
    chunk_id: uuid.UUID
    document_id: uuid.UUID
    document_title: str
    ordinal: int
    text: str
    section_path: Optional[str]
    page_start: int
```

```
    page_end: int
    rank: int
    bm25_score: Optional[float]
    vector_score: Optional[float]
    fused_score: Optional[float]
    rerank_score: Optional[float]

class SearchResponse(BaseModel):
    query_id: uuid.UUID
    results: List[RetrievedChunk]
    latency_ms: int
```

```
# apps/api/src/docintel/schemas/answer.py
from typing import List, Optional, Literal
import uuid
from pydantic import BaseModel, Field
from .search import RetrievedChunk

class AnswerRequest(BaseModel):
    query: str = Field(min_length=1, max_length=2000)
    top_k: int = Field(default=8, ge=1, le=20)
    strategy: Literal["vector_only", "bm25_only", "hybrid", "hybrid_reranked"] =
"hybrid_reranked"
    model: Optional[str] = None  # any OpenRouter-hosted model ID (e.g. "anthropic/claude-
haiku-4-5", "openai/gpt-4o-mini"); default: settings.default_generation_model
    temperature: float = Field(default=0.0, ge=0.0, le=1.0)
    max_tokens: int = Field(default=1024, ge=64, le=4096)

class CitationOut(BaseModel):
    ordinal: int
    chunk_id: uuid.UUID
    document_id: uuid.UUID
    document_title: str
    page_start: int
    page_end: int
    section_path: Optional[str]
    span_text: str

class AnswerResponse(BaseModel):
    query_id: uuid.UUID
    answer_id: uuid.UUID
    answer: str
    citations: List[CitationOut]
    contexts: List[RetrievedChunk]
    model: str
    prompt_tokens: int
    completion_tokens: int
    cost_usd: float
    latency_ms: int
```

```python
# apps/api/src/docintel/schemas/eval.py
from typing import List, Optional, Literal
from datetime import datetime
import uuid
from pydantic import BaseModel

class EvalRunCreate(BaseModel):
    suite_version: str = "v1"
    retrieval_strategy: Literal["vector_only", "bm25_only", "hybrid", "hybrid_reranked"] = "hybrid_reranked"
    generation_model: Optional[str] = None  # any OpenRouter-hosted model ID; default: settings.default_generation_model
    judge_model: Optional[str] = None        # any OpenRouter-hosted model ID; default: settings.default_judge_model (routed through OpenRouter)
    fail_fast: bool = False

class EvalRunOut(BaseModel):
    id: uuid.UUID
    suite_version: str
    git_sha: Optional[str]
    generation_model: str
    judge_model: str
    retrieval_strategy: str
    status: str
    total_cases: int
    cases_passed: int
    faithfulness_mean: Optional[float]
    context_precision_mean: Optional[float]
    context_recall_mean: Optional[float]
    answer_relevancy_mean: Optional[float]
    thresholds_json: dict
    started_at: datetime
    finished_at: Optional[datetime]
```

```python
# apps/api/src/docintel/schemas/drift.py
from typing import Optional
from datetime import datetime
import uuid
from pydantic import BaseModel

class DriftReportCreate(BaseModel):
    window_days: int = 7
    reference_window_days: int = 7

class DriftReportOut(BaseModel):
    id: uuid.UUID
    window_start: datetime
    window_end: datetime
    reference_window_start: datetime
    reference_window_end: datetime
```

```
    embedding_drift_score: Optional[float]
    query_drift_score: Optional[float]
    retrieval_quality_delta: Optional[float]
    status: str
    html_path: Optional[str]
    created_at: datetime
```

### Alembic Migration Plan

| Migration ID | File | Tables / Columns Added | Depends On |
|---|---|---|---|
| 001 | `001_foundation.py` | Enables `vector` extension, creates enum types `document_status`, `retrieval_strategy`, `eval_run_status`, `drift_status` | None |
| 002 | `002_documents_and_chunks.py` | `documents`, `chunks` (with HNSW vector index, GIN tsvector index, unique sha256 index) | 001 |
| 003 | `003_queries_retrievals_answers.py` | `queries`, `retrievals`, `answers`, `citations` | 002 |
| 004 | `004_eval_runs_and_cases.py` | `eval_runs`, `eval_cases` | 003 |
| 005 | `005_drift_reports.py` | `drift_reports` | 004 |

---

## 5. API Contracts

### Base URL

- Development: `http://localhost:8000/api/v1`
- All routes prefixed with `/api/v1`
- All IDs are UUIDs
- All timestamps are ISO 8601 UTC

### Authentication

**Strategy:** API Key in `X-API-Key` header for v1.

```
X-API-Key: <key>
```

The API loads keys from the `API_KEYS` env var (comma-separated). All write endpoints require a valid key. `/health/*` and `/metrics` are unauthenticated. Resolved on 2026-04-13: API-key auth is the committed v1 scheme; JWT is an explicit non-goal until a public web UI is added.

### Standard Response Envelope (paginated)

```
{ "data": [], "meta": { "page": 1, "per_page": 50, "total": 124 } }
```

### Standard Error Format

```
{ "error": { "code": "RESOURCE_NOT_FOUND", "message": "Document not found", "detail": {} } }
```

## Endpoint Definitions

### Health

`GET /api/v1/health/liveness`

**Description:** Process is alive. Unauthenticated. **Success (200):** `{"status": "ok"}`

`GET /api/v1/health/readiness`

**Description:** DB reachable. Unauthenticated. **Success (200):** `{"status": "ok", "db": "connected", "vector_extension": true}` **Errors:** `503 SERVICE_UNAVAILABLE` if DB unreachable or `vector` extension missing.

### Documents

`POST /api/v1/documents` **(multipart)**

**Description:** Upload a PDF and trigger ingestion synchronously (small docs) or in background (large). **Form fields:** `file` (PDF), `title` (string, optional), `source_uri` (string, optional) **Success (201):**

```
{ "id": "uuid", "title": "EU AI Act", "sha256": "...", "page_count": 144, "status": "ingesting", "created_at": "..." }
```

**Errors:**

- `400 INVALID_FILE` : not a PDF / unreadable
- `409 DUPLICATE_DOCUMENT` : same sha256 already ingested
- `413 PAYLOAD_TOO_LARGE` : > `MAX_UPLOAD_BYTES`

`GET /api/v1/documents`

**Query:** `page` (default 1), `per_page` (default 50, max 200), `status` (optional) **Success (200):** `Paginated<DocumentOut>`

`GET /api/v1/documents/{id}`

**Success (200):** `DocumentOut` (includes `chunk_count` ) **Errors:** `404 NOT_FOUND`

`DELETE /api/v1/documents/{id}`

**Success (204):** no body. Cascade-deletes chunks and dependent retrievals. **Errors:** `404 NOT_FOUND`

`POST /api/v1/documents/{id}/reingest`

**Description:** Re-runs chunking + embedding (e.g., after model change). **Success (202):** `{"id": "uuid", "status": "ingesting"}` **Errors:** `404 NOT_FOUND` , `409 DOCUMENT_BUSY`

### Search

`POST /api/v1/search`

**Description:** Hybrid retrieval, no LLM. Persists `queries` + `retrievals` rows. **Body:** `SearchRequest` **Success (200):** `SearchResponse` **Errors:**

- `422 UNPROCESSABLE_ENTITY` : bad request body
- `503 EMBEDDING_MODEL_UNAVAILABLE` : model not loaded

### Answer

`POST /api/v1/answer`

**Description:** Hybrid retrieval + LLM generation with extracted citations. Persists `queries` + `retrievals` + `answers` + `citations`. **Body:** `AnswerRequest` **Success (200):** `AnswerResponse` **Errors:**

- `422 UNPROCESSABLE_ENTITY`
- `502 LLM_PROVIDER_ERROR` : upstream failure
- `503 EMBEDDING_MODEL_UNAVAILABLE`

**Evaluation**

`POST /api/v1/eval/runs`

**Description:** Kicks off a RAGAS run over the v1 fixture. Returns immediately with `RUNNING` status; runs in BackgroundTasks. **Body:** `EvalRunCreate` **Success (202):** `EvalRunOut` **Errors:** `503 LLM_PROVIDER_ERROR` (judge unreachable)

`GET /api/v1/eval/runs`

**Query:** `page` , `per_page` , `status` **Success (200):** `Paginated<EvalRunOut>`

`GET /api/v1/eval/runs/{id}`

**Success (200):** `EvalRunOut` **Errors:** `404 NOT_FOUND`

`GET /api/v1/eval/runs/{id}/cases`

**Query:** `page` , `per_page` , `passed` (bool, optional) **Success (200):** `Paginated<EvalCaseOut>`

**Drift**

`POST /api/v1/drift/reports`

**Description:** Generates a drift report comparing the last `window_days` vs the prior `reference_window_days` . **Body:** `DriftReportCreate` **Success (202):** `DriftReportOut`

`GET /api/v1/drift/reports`

**Query:** `page` , `per_page` , `status` **Success (200):** `Paginated<DriftReportOut>`

`GET /api/v1/drift/reports/{id}`

**Success (200):** `DriftReportOut` with `html_url` (signed local path) included **Errors:** `404 NOT_FOUND`

**Metrics**

`GET /metrics`

**Description:** Prometheus exposition. Unauthenticated. Mounted at root, not under `/api/v1` . **Success (200):** `text/plain; version=0.0.4`

---

## 6. Phase Plan

### Phase Overview

| Phase | Name | Objective | Complexity |
|---|---|---|---|
| 0 | Brief and Architecture | This blueprint | DONE |
| 1 | Foundation | Repo skeleton, FastAPI app, Postgres+pgvector compose, Alembic 001, health, base tests | Medium |

| | | | |
|---|---|---|---|
| 2 | Ingestion Pipeline | PDF loader, semantic chunker, embedder, document/chunk APIs, migration 002 | High |
| 3 | Retrieval Layer | BM25 (tsvector), pgvector ANN, RRF fusion, cross-encoder rerank, `/search`, migration 003 | High |
| 4 | Generation & Citations | LLM client, citation-grounded prompt, citation extractor, `/answer` | Medium |
| 5 | Evaluation Harness | RAGAS runner, golden 100-case fixture, `/eval/*` endpoints, CLI, GitHub Actions gate, migration 004 | High |
| 6 | Observability | LangSmith middleware, structlog wiring, Prometheus metrics, `/metrics` | Medium |
| 7 | Drift Monitoring | Evidently runner, `/drift/*`, APScheduler weekly job, migration 005 | Medium |
| 8 | Streamlit Dashboard | apps/dashboard with eval/drift/cost/explorer pages | Medium |
| 9 | Hardening | Dockerfiles, prod compose, CI workflows, README, deployment docs, release checklist | Medium |

---

### Phase 1: Foundation

**Objective:** Runnable repo skeleton with all infrastructure wired but no domain logic. `docker compose up -d` brings up FastAPI + Postgres+pgvector, health endpoints respond, base tests green.

**Deliverables:**

| File Path | Description |
|---|---|
| `pyproject.toml` (root) | Workspace ruff config + tool settings |
| `apps/api/pyproject.toml` | All Section 2 backend deps pinned exactly |
| `apps/api/src/docintel/__init__.py` | Package init |
| `apps/api/src/docintel/main.py` | FastAPI app factory, lifespan, CORS, health router include, `/metrics` mount stub |
| `apps/api/src/docintel/config.py` | `Settings` (BaseSettings) reading env per Section 7 |
| `apps/api/src/docintel/database.py` | Async engine, `AsyncSessionLocal`, `get_db` dep, `check_vector_extension()` |
| `apps/api/src/docintel/logging_setup.py` | structlog config (JSON in prod, console in dev) |
| `apps/api/src/docintel/auth.py` | `require_api_key` dependency reading from `Settings.api_keys` |
| `apps/api/src/docintel/models/base.py` | `Base = DeclarativeBase` |
| `apps/api/src/docintel/routers/health.py` | `/health/liveness`, `/health/readiness` |
| `apps/api/src/docintel/schemas/common.py` | Error envelope + Paginated generic |

| | |
|---|---|
| `apps/api/alembic.ini` | Alembic config pointing at `apps/api/alembic` |
| `apps/api/alembic/env.py` | Async env reading `DATABASE_URL` from settings |
| `apps/api/alembic/script.py.mako` | Default Alembic template |
| `apps/api/alembic/versions/001_foundation.py` | Enables `CREATE EXTENSION IF NOT EXISTS vector`; creates enum types from Section 4 |
| `apps/api/tests/conftest.py` | Async test client + in-memory SQLite session fixture (vector-extension-dependent tests skipped on SQLite) |
| `apps/api/tests/test_health.py` | `test_liveness`, `test_readiness_ok`, `test_readiness_db_down` |
| `apps/api/Dockerfile` | Multi-stage Python 3.12-slim image with uv |
| `docker-compose.yml` | Dev compose: `api` + `db` (pgvector/pgvector:pg16) |
| `ops/docker/postgres-init/001_pgvector.sql` | `CREATE EXTENSION IF NOT EXISTS vector;` |
| `.env.example` | All env keys from Section 7 with placeholders |
| `.gitignore` | Python, Node, Docker, IDE, model cache, artifacts |
| `CLAUDE.md` | From Section 10 template |
| `docs/HANDOFF.md` | Phase 1 status; Phase 2 next |
| `docs/PROGRESS.md` | Phase 1 marked complete after verification |
| `docs/DECISIONS.md` | Copies Section 12 decision log |
| `docs/architecture.md` | Copies Section 1 diagram + responsibilities |
| `docs/verification.md` | Phase 1 verification commands |

**Endpoints in this phase:**

- `GET /api/v1/health/liveness` → 200 `{"status": "ok"}`
- `GET /api/v1/health/readiness` → 200 with DB + vector status, 503 if unavailable

**Test Requirements:**

- `test_liveness` : 200 + `{"status": "ok"}`
- `test_readiness_ok` : 200 against in-memory SQLite (skip vector check on SQLite)
- `test_readiness_db_down` : 503 with disconnected session override

**Verification Commands:**

```
cd "apps/api"
uv sync
uv run alembic upgrade head
uv run pytest tests/test_health.py -v
cd ../..
docker compose up -d
```

```
curl http://localhost:8000/api/v1/health/liveness
curl http://localhost:8000/api/v1/health/readiness
```

**Definition of Done:**

- [ ] All deliverables exist with non-stub implementations
- [ ] All four verification commands succeed
- [ ] `docs/PROGRESS.md`, `docs/HANDOFF.md`, `CLAUDE.md` updated

---

## Phase 2: Ingestion Pipeline

**Objective:** Upload a PDF, parse it, chunk it semantically, embed it with bge-small-en-v1.5, persist documents + chunks (with HNSW + GIN indexes). End state: the EU AI Act PDF is fully ingested via the CLI and visible via `GET /documents/{id}`.

**Deliverables:**

| File Path | Description |
|---|---|
| `apps/api/src/docintel/models/document.py` | Per Section 4 |
| `apps/api/src/docintel/models/chunk.py` | Per Section 4 (with Vector + tsvector) |
| `apps/api/src/docintel/schemas/document.py` | `DocumentCreate`, `DocumentOut`, `DocumentList` |
| `apps/api/src/docintel/services/ingestion/pdf_loader.py` | `load_pdf(path) -> List[PageText]` using pypdf; preserves page numbers + heading hints |
| `apps/api/src/docintel/services/ingestion/chunker.py` | `chunk_pages(pages, target_tokens=512, overlap_tokens=64) -> List[ChunkDraft]`; heading-aware splitter |
| `apps/api/src/docintel/services/ingestion/embedder.py` | `Embedder` singleton wrapping `SentenceTransformer("BAAI/bge-small-en-v1.5")`; batched `embed_texts` |
| `apps/api/src/docintel/services/ingestion/pipeline.py` | `ingest_document(file_bytes, title, source_uri) -> Document` orchestrator |
| `apps/api/src/docintel/routers/documents.py` | All `/documents/*` endpoints from Section 5 |
| `apps/api/alembic/versions/002_documents_and_chunks.py` | Creates `documents`, `chunks` with all indexes from Section 4 |
| `apps/api/src/docintel/tools/ingest_eu_ai_act.py` | CLI that downloads (if `EU_AI_ACT_PDF_URL` set) or reads local path and calls pipeline |
| `apps/api/tests/test_chunker.py` | golden chunk-boundary tests on synthetic 3-page input |
| `apps/api/tests/test_embedder.py` | embeds 2 texts; asserts shape (2, 384) and unit-norm |
| `apps/api/tests/test_documents.py` | upload tiny_pdf.pdf → 201; duplicate → 409; list/get/delete |

| | |
|---|---|
| `apps/api/tests/fixtures/tiny_pdf.pdf` | 3-page synthetic PDF |
| `data/source/README.md` | EU AI Act PDF source URL + sha256 expected |
| `docs/ingestion.md` | Pipeline diagram + chunker rationale |

**Migration:** `002_documents_and_chunks.py` — adds `documents`, `chunks` (with HNSW vector_cosine_ops index and GIN tsvector index).

**Test Requirements:**

- `test_chunker`: deterministic chunk count + ordinals on fixed input
- `test_embedder`: shape + L2 norm tolerance
- `test_documents_upload_pdf_201`: returns id + sha256 + page_count
- `test_documents_upload_duplicate_409`: same bytes → 409
- `test_documents_list_pagination`
- `test_documents_get_404`
- `test_documents_delete_204_cascades_chunks`

**Verification Commands:**

```
docker compose up -d
cd apps/api
uv run alembic upgrade head
uv run pytest tests/test_chunker.py tests/test_embedder.py tests/test_documents.py -v
# Real PDF (set EU_AI_ACT_PDF_URL in .env or pass --path)
uv run python -m docintel.tools.ingest_eu_ai_act --path "<local pdf>"
curl -H "X-API-Key: $API_KEY" http://localhost:8000/api/v1/documents
```

**Definition of Done:**

- [ ] All deliverables exist
- [ ] All test cases pass
- [ ] EU AI Act PDF ingests end-to-end and `chunks` row count > 0
- [ ] `docs/PROGRESS.md`, `docs/HANDOFF.md`, `CLAUDE.md` updated

---

## Phase 3: Retrieval Layer

**Objective:** Hybrid retrieval works end-to-end: BM25 over tsvector, ANN over pgvector, Reciprocal Rank Fusion, cross-encoder rerank. `/search` returns ranked chunks with all score components, persists `queries` + `retrievals`.

**Deliverables:**

| File Path | Description |
|---|---|
| `apps/api/src/docintel/models/query.py` | Per Section 4 (incl. `RetrievalStrategy` enum) |
| `apps/api/src/docintel/models/retrieval.py` | Per Section 4 |
| `apps/api/src/docintel/schemas/search.py` | Per Section 5 |
| `apps/api/src/docintel/services/retrieval/bm25.py` | `bm25_search(session, query, top_k)` using `ts_rank_cd(tsv, plainto_tsquery('english', :q))` |

| | |
|---|---|
| apps/api/src/docintel/services/retrieval/vector.py | vector_search(session, query_vec, top_k) using embedding <=> :vec cosine |
| apps/api/src/docintel/services/retrieval/fusion.py | reciprocal_rank_fusion(lists, k=60) -> ranked List[ChunkScore] |
| apps/api/src/docintel/services/retrieval/reranker.py | Reranker singleton wrapping CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2") |
| apps/api/src/docintel/services/retrieval/hybrid.py | hybrid_search(query, strategy, top_k, rerank_top_n, rrf_k) -> SearchResult |
| apps/api/src/docintel/routers/search.py | POST /search |
| apps/api/alembic/versions/003_queries_retrievals_answers.py | Creates queries, retrievals, answers, citations (answers/citations populated in Phase 4 but tables created here for atomic migration order) |
| apps/api/src/docintel/tools/benchmark_retrieval.py | CLI: runs all 4 strategies on fixture and prints precision@k, recall@k |
| apps/api/tests/test_bm25.py | seeds 5 chunks, asserts term-match top-1 |
| apps/api/tests/test_vector.py | seeds 5 chunks, asserts nearest-vec top-1 |
| apps/api/tests/test_fusion.py | RRF unit test with handcrafted ranked lists |
| apps/api/tests/test_reranker.py | rerank changes order in expected way on 3 candidates |
| apps/api/tests/test_search_endpoint.py | end-to-end against seeded DB; verifies score components present per strategy |
| docs/retrieval.md | Strategy table, RRF formula, reranker contract |

**Migration:** `003_queries_retrievals_answers.py` — adds the 4 tables.

**Test Requirements:**

- All four strategies return non-empty results on seeded data
- `vector_only` populates `vector_score` only; `bm25_only` populates `bm25_score` only; `hybrid` populates both + `fused_score`; `hybrid_reranked` adds `rerank_score`
- `test_search_persists_query_and_retrievals`: rows present after request
- `test_search_invalid_strategy_422`

**Verification Commands:**

```
docker compose up -d
cd apps/api
uv run alembic upgrade head
uv run pytest tests/test_bm25.py tests/test_vector.py tests/test_fusion.py
tests/test_reranker.py tests/test_search_endpoint.py -v
uv run python -m docintel.tools.benchmark_retrieval --top-k 10
curl -X POST http://localhost:8000/api/v1/search -H "X-API-Key: $API_KEY" -H "Content-Type:
```

```
application/json" -d '{"query":"What is a high-risk AI
system?","strategy":"hybrid_reranked","top_k":5}'
```

**Definition of Done:**

- [ ] All four strategies returning expected score components
- [ ] Benchmark CLI shows hybrid_reranked beating vector_only on seeded data
- [ ] All tests pass
- [ ] Continuity docs updated

---

## Phase 4: Generation & Citations

**Objective:** `/answer` returns a citation-grounded answer. Persists `answers` + `citations`. Cost and latency tracked per request.

**Deliverables:**

| File Path | Description |
|---|---|
| `apps/api/src/docintel/models/answer.py` | Per Section 4 |
| `apps/api/src/docintel/models/citation.py` | Per Section 4 |
| `apps/api/src/docintel/schemas/answer.py` | Per Section 5 |
| `apps/api/src/docintel/services/generation/prompt.py` | System + user template requiring `[c#N]` markers |
| `apps/api/src/docintel/services/generation/llm_client.py` | `OpenRouterClient` (httpx.AsyncClient) with retries + cost lookup table |
| `apps/api/src/docintel/services/generation/citation_extractor.py` | Parses `[c#N]` from LLM output, maps to retrieved chunks |
| `apps/api/src/docintel/services/generation/answerer.py` | Orchestrator: retrieval → prompt → LLM → citations → persist |
| `apps/api/src/docintel/routers/answer.py` | `POST /answer` |
| `apps/api/tests/test_citation_extractor.py` | unit tests for parser edge cases |
| `apps/api/tests/test_answer_endpoint.py` | integration test using a stubbed `OpenRouterClient` (httpx mock) |
| `docs/api.md` | Updated with /answer examples |

**Test Requirements:**

- `test_citation_extractor_parses_markers`
- `test_citation_extractor_drops_unknown_markers`
- `test_answer_endpoint_returns_citations_with_chunk_metadata`
- `test_answer_persists_query_retrievals_answer_citations`
- `test_answer_llm_502_on_provider_error`

**Verification Commands:**

```
docker compose up -d
cd apps/api
uv run pytest tests/test_citation_extractor.py tests/test_answer_endpoint.py -v
# Live call (requires OPENROUTER_API_KEY in .env)
curl -X POST http://localhost:8000/api/v1/answer -H "X-API-Key: $API_KEY" -H "Content-Type: application/json" -d '{"query":"Define high-risk AI system per the EU AI Act","top_k":6}'
```

**Definition of Done:**

- [ ] All tests pass
- [ ] Live `/answer` call returns answer + at least 1 citation backed by a real chunk
- [ ] Continuity docs updated

---

## Phase 5: Evaluation Harness

**Objective:** RAGAS pipeline runs over the 100-case golden fixture, persists per-case scores + run aggregates, and gates GitHub Actions PRs against thresholds.

**Deliverables:**

| File Path | Description |
|---|---|
| `apps/api/src/docintel/models/eval_run.py` | Per Section 4 |
| `apps/api/src/docintel/models/eval_case.py` | Per Section 4 |
| `apps/api/src/docintel/schemas/eval.py` | Per Section 5 |
| `apps/api/src/docintel/services/evaluation/fixture_loader.py` | Loads + JSON-schema-validates `fixtures/eu_ai_act_qa_v1.json` |
| `apps/api/src/docintel/services/evaluation/ragas_runner.py` | Runs RAGAS over the answerer; uses `langchain-openai` against OpenRouter for the judge |
| `apps/api/src/docintel/services/evaluation/thresholds.py` | Defaults: faithfulness>=0.85, context_precision>=0.88, context_recall>=0.80, answer_relevancy>=0.85; per-case pass = all four |
| `apps/api/src/docintel/services/evaluation/ci_gate.py` | CLI: runs latest fixture, exits non-zero if any aggregate below threshold |
| `apps/api/src/docintel/routers/eval.py` | `/eval/runs`, `/eval/runs/{id}`, `/eval/runs/{id}/cases` |
| `apps/api/src/docintel/tools/run_eval.py` | Same as ci_gate but with rich console output |
| `apps/api/alembic/versions/004_eval_runs_and_cases.py` | Creates `eval_runs`, `eval_cases` |

| | |
|---|---|
| `fixtures/eu_ai_act_qa_v1.json` | 25 Q&A pairs, LLM-seeded from the EU AI Act PDF and manually reviewed, tagged `version: "v0.1"`. Resolved on 2026-04-13: ship v0.1 now; a follow-up issue tracks expansion to 100 cases v1.0. |
| `apps/api/src/docintel/tools/seed_fixture.py` | CLI that generates seed Q&A pairs from ingested chunks using the configured generation model; emits a pre-filled `fixtures/eu_ai_act_qa_v1.json` for human review |
| `fixtures/ISSUES.md` | Open question queue for reviewer edits; links follow-up: grow v0.1 → v1.0 (100 cases) |
| `fixtures/eu_ai_act_qa_v1.schema.json` | JSON schema for fixture |
| `fixtures/README.md` | Curation method + add-a-case guide |
| `apps/api/tests/test_eval_runner.py` | tiny 3-case fixture + stubbed judge → asserts persisted scores match expected |
| `.github/workflows/ragas-eval.yml` | PR-triggered job: spins up postgres, ingests sample, runs ci_gate; needs `OPENROUTER_API_KEY` repo secret |
| `docs/evaluation.md` | Threshold table, judge model, fixture spec |

**Migration:** `004_eval_runs_and_cases.py`.

**Endpoints:** `/eval/runs` (POST/GET), `/eval/runs/{id}` (GET), `/eval/runs/{id}/cases` (GET).

**Test Requirements:**

- `test_fixture_loader_validates_schema`
- `test_ragas_runner_persists_run_and_cases_with_stubbed_judge`
- `test_eval_endpoints_pagination`
- `test_ci_gate_exits_nonzero_on_threshold_breach`

**Verification Commands:**

```
docker compose up -d
cd apps/api
uv run alembic upgrade head
uv run pytest tests/test_eval_runner.py -v
# Real judge run (requires OPENROUTER_API_KEY)
uv run python -m docintel.tools.run_eval --suite-version v1 --strategy hybrid_reranked
# CI gate (used by GH Action)
uv run python -m docintel.services.evaluation.ci_gate --fail-on-breach
```

**ENV REQUIRED:**

- `OPENROUTER_API_KEY` — used by judge and (optionally) by `/answer`

**Definition of Done:**

- [ ] All tests pass
- [ ] Real eval run completes, persists run + N cases with non-null scores
- [ ] GitHub Actions workflow runs successfully on a PR (or dry-run logged)
- [ ] Continuity docs updated

---

### Phase 6: Observability

**Objective:** Every request is traced (LangSmith optional), structured-logged, and counted in Prometheus metrics. `/metrics` exposes the standard set.

**Deliverables:**

| File Path | Description |
|---|---|
| `apps/api/src/docintel/services/monitoring/langsmith_setup.py` | Initializes LangSmith client if `LANGSMITH_API_KEY` set; no-op otherwise |
| `apps/api/src/docintel/services/monitoring/metrics.py` | Prometheus collectors: `docintel_requests_total`, `docintel_request_duration_seconds`, `docintel_retrieval_score`, `docintel_llm_tokens_total`, `docintel_llm_cost_usd_total`, `docintel_eval_score{metric}` |
| `apps/api/src/docintel/services/monitoring/tracing.py` | FastAPI middleware: request_id, latency, status code, log line + counter |
| `apps/api/src/docintel/routers/metrics.py` | `GET /metrics` (mounted at root) |
| `apps/api/tests/test_metrics.py` | hits `/metrics`, asserts collector names present |
| `apps/api/tests/test_tracing_middleware.py` | request id propagation + latency recording |
| `docs/observability.md` | Metric catalog + LangSmith setup |

**Test Requirements:**

- `test_metrics_exposes_collectors`
- `test_request_middleware_sets_request_id_and_records_latency`

**Verification Commands:**

```
docker compose up -d
cd apps/api
uv run pytest tests/test_metrics.py tests/test_tracing_middleware.py -v
curl http://localhost:8000/metrics | grep docintel_
# Generate one /search to populate counters then re-curl /metrics
```

**Definition of Done:**

- [ ] All tests pass
- [ ] `/metrics` shows non-zero counters after one search
- [ ] If LangSmith key set, traces appear in LangSmith project (manual check)

- [ ] Continuity docs updated

---

### Phase 7: Drift Monitoring

**Objective:** Evidently-powered weekly drift report comparing the last 7 days of queries+retrievals to the prior 7 days. Reports persisted + browsable. APScheduler runs the job in-process every Monday 02:00 UTC.

**Deliverables:**

| File Path | Description |
|---|---|
| `apps/api/src/docintel/models/drift_report.py` | Per Section 4 |
| `apps/api/src/docintel/schemas/drift.py` | Per Section 5 |
| `apps/api/src/docintel/services/drift/evidently_runner.py` | Builds Evidently `Report` for: query embedding drift (current vs reference), retrieval rank-stability, mean rerank score delta |
| `apps/api/src/docintel/services/drift/reporter.py` | Persists `drift_reports` row + writes HTML to `ARTIFACT_STORAGE_PATH/drift/<id>.html` |
| `apps/api/src/docintel/services/drift/scheduler.py` | APScheduler AsyncIOScheduler started in app lifespan; weekly cron Monday 02:00 UTC |
| `apps/api/src/docintel/routers/drift.py` | `/drift/reports` endpoints |
| `apps/api/src/docintel/tools/run_drift.py` | CLI: one-shot drift report |
| `apps/api/alembic/versions/005_drift_reports.py` | Creates `drift_reports` |
| `apps/api/tests/test_drift_runner.py` | seeds two windows of fake queries; asserts non-null drift score + status thresholds |
| `docs/drift.md` | What's compared, thresholds (warning at 0.15, alert at 0.25), HTML report layout |

**Migration:** `005_drift_reports.py`.

**Test Requirements:**

- `test_drift_runner_computes_scores_on_seeded_data`
- `test_drift_endpoints_pagination`
- `test_drift_status_warning_at_threshold` / `test_drift_status_alert_above_threshold`

**Verification Commands:**

```
docker compose up -d
cd apps/api
uv run alembic upgrade head
uv run pytest tests/test_drift_runner.py -v
uv run python -m docintel.tools.run_drift --window-days 7 --reference-window-days 7
curl -H "X-API-Key: $API_KEY" http://localhost:8000/api/v1/drift/reports
```

**Definition of Done:**

- [ ] All tests pass
- [ ] One-shot CLI emits a row + HTML artifact
- [ ] Scheduler shows registered job in app startup logs
- [ ] Continuity docs updated

---

### Phase 8: Streamlit Dashboard

**Objective:** Read-only ops dashboard with four pages: eval trends, drift reports, cost & latency, retrieval explorer. Deployed alongside API in compose.

**Deliverables:**

| File Path | Description |
|---|---|
| `apps/dashboard/pyproject.toml` | Streamlit + plotly + sqlalchemy + psycopg pinned per Section 2 |
| `apps/dashboard/app.py` | Home page: KPI tiles (latest faithfulness, p95 latency, drift status, total cost last 7d) |
| `apps/dashboard/lib/db.py` | Sync Postgres engine for read-only queries |
| `apps/dashboard/lib/api_client.py` | httpx wrapper with API key |
| `apps/dashboard/pages/1_Eval_Trends.py` | Line chart of faithfulness/precision/recall/relevancy across runs |
| `apps/dashboard/pages/2_Drift_Reports.py` | Table of reports + embed HTML viewer |
| `apps/dashboard/pages/3_Cost_and_Latency.py` | Cost per day, p50/p95 latency by endpoint, model breakdown |
| `apps/dashboard/pages/4_Retrieval_Explorer.py` | Type a query → live `/search` call → side-by-side ranked chunks per strategy |
| `apps/dashboard/Dockerfile` | Python 3.12-slim Streamlit image |
| `ops/docker/compose.full.yml` | Adds `dashboard` service on port 8501 |
| `docs/deployment.md` | Updated with dashboard URL |

**Test Requirements:**

- Manual smoke: each page loads, data populated after a few requests
- `apps/dashboard/tests/test_db_queries.py` : query helper functions return expected shape against seeded DB

**Verification Commands:**

```
docker compose -f docker-compose.yml -f ops/docker/compose.full.yml up -d
# Generate some traffic
curl -X POST http://localhost:8000/api/v1/search ...
curl -X POST http://localhost:8000/api/v1/answer ...
uv run python -m docintel.tools.run_eval
# Visit dashboard
start http://localhost:8501
```

**Definition of Done:**

- [ ] Compose with dashboard up; all four pages render with non-empty data after seeding
- [ ] Continuity docs updated

---

## Phase 9: Hardening

**Objective:** Production-shape Dockerfiles, CI workflows, complete README, deployment doc, release checklist. Repo is ready to share publicly.

**Deliverables:**

| File Path | Description |
|---|---|
| `apps/api/Dockerfile` | Already exists from Phase 1; harden: non-root user, healthcheck, model cache volume |
| `apps/dashboard/Dockerfile` | Already exists from Phase 8; harden similarly |
| `ops/docker/compose.full.yml` | Final: api + db + dashboard + named volumes for pgdata and model cache |
| `docker-compose.prod.yml` | Production overlay: pinned image tags, restart policies, resource limits |
| `.github/workflows/ci.yml` | Lint (ruff) + type-check + unit tests on every PR |
| `.github/workflows/ragas-eval.yml` | Already exists from Phase 5; finalize with caching for embedding model |
| `README.md` | Project tagline, architecture diagram, quickstart, screenshots, KPI table, links |
| `docs/deployment.md` | Compose, env, model cache, scaling notes |
| `docs/verification.md` | All phase verification commands consolidated |
| `docs/release_checklist.md` | Pre-push checklist (tests, eval, drift sanity, docs) |
| `LICENSE` | Already present |

**Test Requirements:**

- `ci.yml` green on a sample PR
- `ragas-eval.yml` green on a sample PR

**Verification Commands:**

```
docker compose -f docker-compose.yml -f docker-compose.prod.yml config
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d
gh workflow run ci.yml --ref main
gh workflow run ragas-eval.yml --ref main
```

**Definition of Done:**

- [ ] Both workflows green
- [ ] Prod compose validates and runs
- [ ] README complete with verified commands

- [ ] All docs reflect verified state
- [ ] `CLAUDE.md`, `docs/PROGRESS.md`, `docs/HANDOFF.md` show project complete

---

## 7. Environment Variables

```
# --- Core ---
DATABASE_URL=postgresql+asyncpg://postgres:postgres@db:5432/docintel
# Dev/test override: sqlite+aiosqlite:///./local.db (note: vector + tsvector tests skipped
on SQLite)

API_PORT=8000
DEBUG=false
LOG_LEVEL=INFO
LOG_FORMAT=json                    # json|console

# --- Auth ---
API_KEYS=dev-key-change-me         # comma-separated; ENV REQUIRED in prod
SECRET_KEY=                        # generate: python -c "import secrets;
print(secrets.token_hex(32))"

# --- Models ---
EMBEDDING_MODEL=BAAI/bge-small-en-v1.5
EMBEDDING_MODEL_REVISION=          # pin a HF revision before going to prod
RERANKER_MODEL=cross-encoder/ms-marco-MiniLM-L-6-v2
RERANKER_MODEL_REVISION=
MODEL_CACHE_DIR=/app/.model_cache

# --- LLM (OpenRouter) ---
OPENROUTER_API_KEY=                # ENV REQUIRED for /answer + RAGAS judge
OPENROUTER_BASE_URL=https://openrouter.ai/api/v1
DEFAULT_GENERATION_MODEL=anthropic/claude-haiku-4-5
DEFAULT_JUDGE_MODEL=openai/gpt-4o-mini

# --- LangSmith (optional) ---
LANGSMITH_API_KEY=                 # optional; if unset tracing is disabled
LANGSMITH_PROJECT=docintel-dev
LANGSMITH_TRACING=false

# --- Storage ---
ARTIFACT_STORAGE_PATH=/app/artifacts

# --- Ingestion ---
MAX_UPLOAD_BYTES=52428800          # 50 MiB
CHUNK_TARGET_TOKENS=512
CHUNK_OVERLAP_TOKENS=64
EU_AI_ACT_PDF_URL=                 # optional; CLI helper

# --- Retrieval ---
DEFAULT_TOP_K=10
DEFAULT_RERANK_TOP_N=50
```

```
DEFAULT_RRF_K=60

# --- Eval ---
EVAL_FAITHFULNESS_THRESHOLD=0.85
EVAL_CONTEXT_PRECISION_THRESHOLD=0.88
EVAL_CONTEXT_RECALL_THRESHOLD=0.80
EVAL_ANSWER_RELEVANCY_THRESHOLD=0.85

# --- Drift ---
DRIFT_WINDOW_DAYS=7
DRIFT_REFERENCE_WINDOW_DAYS=7
DRIFT_WARNING_THRESHOLD=0.15
DRIFT_ALERT_THRESHOLD=0.25
DRIFT_CRON=0 2 * * 1               # Mondays 02:00 UTC
```

`Settings` pattern (Phase 1):

```python
from typing import List
from pydantic import Field, AnyUrl
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    database_url: str
    api_port: int = 8000
    debug: bool = False
    log_level: str = "INFO"
    log_format: str = "json"
    api_keys: List[str] = Field(default_factory=list)
    secret_key: str = "dev-secret-not-for-production"

    embedding_model: str = "BAAI/bge-small-en-v1.5"
    embedding_model_revision: str | None = None
    reranker_model: str = "cross-encoder/ms-marco-MiniLM-L-6-v2"
    reranker_model_revision: str | None = None
    model_cache_dir: str = "/app/.model_cache"

    openrouter_api_key: str | None = None
    openrouter_base_url: str = "https://openrouter.ai/api/v1"
    default_generation_model: str = "anthropic/claude-haiku-4-5"
    default_judge_model: str = "openai/gpt-4o-mini"

    langsmith_api_key: str | None = None
    langsmith_project: str = "docintel-dev"
    langsmith_tracing: bool = False

    artifact_storage_path: str = "/app/artifacts"

    max_upload_bytes: int = 52_428_800
    chunk_target_tokens: int = 512
    chunk_overlap_tokens: int = 64
    eu_ai_act_pdf_url: str | None = None
```

```
    default_top_k: int = 10
    default_rerank_top_n: int = 50
    default_rrf_k: int = 60

    eval_faithfulness_threshold: float = 0.85
    eval_context_precision_threshold: float = 0.88
    eval_context_recall_threshold: float = 0.80
    eval_answer_relevancy_threshold: float = 0.85

    drift_window_days: int = 7
    drift_reference_window_days: int = 7
    drift_warning_threshold: float = 0.15
    drift_alert_threshold: float = 0.25
    drift_cron: str = "0 2 * * 1"

    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8",
case_sensitive=False)

settings = Settings()
```

---

## 8. Testing Strategy

### Test Pyramid

| Layer | Tool | Coverage Target | What Gets Tested |
|---|---|---|---|
| Unit | pytest | 80%+ of services | chunker boundaries, RRF math, citation parser, threshold logic |
| Integration | pytest + AsyncClient | All endpoints | request/response contracts, auth, error envelopes, persistence |
| Eval | RAGAS via `tools.run_eval` | All 4 metrics | faithfulness, context precision/recall, answer relevancy |
| Benchmark | `tools.benchmark_retrieval` | Strategy comparison | precision@k, recall@k for vector vs hybrid vs reranked |

### Conventions

- All tests use `pytest` with `pytest-asyncio` ; `asyncio_mode = "auto"`
- Test DB: in-memory SQLite for unit tests that don't need pgvector/tsvector; ephemeral Postgres container (via testcontainers) for integration tests that need them — fixture in `conftest.py` selects automatically
- **No mocking the database** — all tests hit a real DB (SQLite or Postgres)
- Mock only: outbound HTTP to OpenRouter, LangSmith, and HuggingFace model downloads (use a tiny stub embedder for unit tests; real model loaded only in integration session-scope fixture)

### conftest.py highlights

```python
# apps/api/tests/conftest.py — full implementation in Phase 1
import pytest, pytest_asyncio
from httpx import AsyncClient, ASGITransport
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession, async_sessionmaker

from docintel.main import app
from docintel.database import get_db
from docintel.models.base import Base

TEST_DB_URL = "sqlite+aiosqlite:///:memory:"

@pytest_asyncio.fixture
async def db_session():
    engine = create_async_engine(TEST_DB_URL)
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
    Session = async_sessionmaker(engine, class_=AsyncSession, expire_on_commit=False)
    async with Session() as s:
        yield s
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.drop_all)

@pytest_asyncio.fixture
async def client(db_session):
    app.dependency_overrides[get_db] = lambda: db_session
    async with AsyncClient(transport=ASGITransport(app=app), base_url="http://test") as c:
        yield c
    app.dependency_overrides.clear()
```

### Golden Fixture Format ( `fixtures/eu_ai_act_qa_v1.json` )

```json
{
  "version": "v1",
  "source_doc_sha256": "<sha of the EU AI Act PDF>",
  "cases": [
    {
      "id": "case-001",
      "question": "What is the definition of a high-risk AI system under Article 6?",
      "ground_truth": "An AI system is high-risk under Article 6 if ...",
      "expected_articles": ["Article 6", "Annex III"],
      "category": "definitions"
    }
  ]
}
```

JSON schema in `fixtures/eu_ai_act_qa_v1.schema.json` enforces required keys and types.

---

## 9. Deployment Architecture

### Local Development ( `docker-compose.yml` )

```yaml
services:
  api:
    build:
      context: .
      dockerfile: apps/api/Dockerfile
    ports:
      - "${API_PORT:-8000}:8000"
    environment:
      DATABASE_URL: postgresql+asyncpg://postgres:postgres@db:5432/docintel
      SECRET_KEY: ${SECRET_KEY:-dev-secret-not-for-production}
      API_KEYS: ${API_KEYS:-dev-key-change-me}
      OPENROUTER_API_KEY: ${OPENROUTER_API_KEY:-}
      LANGSMITH_API_KEY: ${LANGSMITH_API_KEY:-}
      MODEL_CACHE_DIR: /app/.model_cache
      ARTIFACT_STORAGE_PATH: /app/artifacts
    env_file: .env
    volumes:
      - model_cache:/app/.model_cache
      - artifacts:/app/artifacts
    depends_on:
      db:
        condition: service_healthy

  db:
    image: pgvector/pgvector:pg16
    environment:
      POSTGRES_DB: docintel
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./ops/docker/postgres-init:/docker-entrypoint-initdb.d
    ports:
      - "5433:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 5s
      retries: 10

volumes:
  pgdata:
  model_cache:
  artifacts:
```

### Full Stack Overlay ( `ops/docker/compose.full.yml` , used in Phase 8+)

Adds:

```yaml
services:
  dashboard:
    build:
      context: .
      dockerfile: apps/dashboard/Dockerfile
    ports:
      - "8501:8501"
    environment:
      DATABASE_URL: postgresql+psycopg://postgres:postgres@db:5432/docintel
      API_BASE_URL: http://api:8000/api/v1
      API_KEY: ${API_KEYS}
    depends_on:
      api:
        condition: service_started
```

`apps/api/Dockerfile`

```dockerfile
FROM python:3.12-slim AS builder
WORKDIR /app
RUN pip install --no-cache-dir uv==0.5.21
COPY apps/api/pyproject.toml ./
RUN uv sync --frozen --no-dev || uv sync --no-dev

FROM python:3.12-slim
RUN useradd --create-home --shell /bin/bash app
WORKDIR /app
COPY --from=builder /app/.venv .venv
COPY apps/api/src ./src
COPY apps/api/alembic ./alembic
COPY apps/api/alembic.ini ./alembic.ini
RUN mkdir -p /app/.model_cache /app/artifacts && chown -R app:app /app
USER app
ENV PATH="/app/.venv/bin:$PATH" PYTHONPATH=/app/src
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=5s --retries=3 CMD curl -fsS
http://localhost:8000/api/v1/health/liveness || exit 1
CMD ["uvicorn", "docintel.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

`apps/dashboard/Dockerfile`

```dockerfile
FROM python:3.12-slim
WORKDIR /app
RUN pip install --no-cache-dir uv==0.5.21
COPY apps/dashboard/pyproject.toml ./
RUN uv sync --frozen --no-dev || uv sync --no-dev
COPY apps/dashboard/ ./
ENV PATH="/app/.venv/bin:$PATH"
```

```
EXPOSE 8501
CMD ["streamlit", "run", "app.py", "--server.address=0.0.0.0", "--server.port=8501"]
```

### Health Check Endpoints (Phase 1)

- `GET /api/v1/health/liveness` — 200 if process is alive
- `GET /api/v1/health/readiness` — 200 if DB reachable + `vector` extension present, 503 otherwise

---

## 10. CLAUDE.md Template

Codex creates `CLAUDE.md` in repo root during Phase 1 with this content:

```
# A2 DocIntel Working Memory

## Project Identity
- Project: `A2 DocIntel — Production RAG with RAGAS Eval, Hybrid Search & Drift Monitoring`
- Product shape: FastAPI service + Streamlit ops dashboard for hybrid retrieval, citation-grounded RAG, RAGAS-gated CI, and Evidently drift monitoring over the EU AI Act PDF.
- Repo mode: flagship, production-grade, phase-wise delivery
- Active branch: main

## Current Commands
docker compose up -d
uv run --directory apps/api alembic upgrade head
uv run --directory apps/api pytest
uv run --directory apps/api uvicorn docintel.main:app --reload --app-dir src
uv run --directory apps/api python -m docintel.tools.ingest_eu_ai_act --path <pdf>
uv run --directory apps/api python -m docintel.tools.run_eval
uv run --directory apps/api python -m docintel.tools.run_drift

## Active Decisions
- Modular monolith FastAPI + Postgres+pgvector + Streamlit dashboard
- Embeddings: BAAI/bge-small-en-v1.5 (384-dim); reranker: cross-encoder/ms-marco-MiniLM-L-6-v2
- BM25 via Postgres tsvector + GIN; vector via pgvector HNSW (cosine)
- Fusion: Reciprocal Rank Fusion (k=60)
- LLM: OpenRouter; default generation `anthropic/claude-haiku-4-5`, judge `openai/gpt-4o-mini`
- Eval: RAGAS faithfulness/context_precision/context_recall/answer_relevancy with CI gate
- Drift: Evidently weekly job via APScheduler

## Current Execution Truth
- Blueprint: complete (BLUEPRINT.md)
- Phase 1 foundation: not started

## Update Rule
Update this file after each verified phase closure together with:
- docs/HANDOFF.md
- docs/PROGRESS.md
- docs/DECISIONS.md
```

```
- docs/architecture.md
- docs/verification.md
```

---

## 11. Risk Register

| # | Risk | Likelihood | Impact | Mitigation |
|---|---|---|---|---|
| R1 | Pinned versions for `ragas`, `evidently`, `langsmith`, or `langchain-openai` may have shifted between blueprint date (2026-04-13) and execution; APIs change between minor releases | Medium | High | Phase 1 first task: `pip index versions ragas evidently langsmith langchain-openai langchain` and confirm pins resolve. If a pinned version is yanked, bump to nearest patch and record in `docs/DECISIONS.md` |
| R2 | RAGAS judge cost on 100-case fixture could exceed budget per CI run | Medium | Medium | Default judge: `openai/gpt-4o-mini`; expose `EVAL_SAMPLE_SIZE` to allow CI to run a 25-case stratified sample on PRs and full 100 nightly |
| R3 | Cross-encoder rerank latency on CPU may push p95 above 4s for top_n=50 | Medium | Medium | Cap default `RERANK_TOP_N=50`; measure in Phase 3 benchmark; document GPU recipe in deployment doc; allow `strategy=hybrid` (no rerank) as fast path |
| R4 | Hand-curated 100-case golden fixture is out-of-scope for an automated build | High | Medium | Ship 25-case v0.1 fixture seeded by an LLM from PDF + manually reviewed; mark `DECISION NEEDED` to confirm whether to grow to 100 v1.0 before claiming success criteria met |
| R5 | pgvector HNSW index build can be memory-heavy on large corpora; first ingest may slow Phase 2 | Low | Medium | Use `m=16, ef_construction=64` defaults (modest memory); document `SET maintenance_work_mem` recipe in `docs/ingestion.md`; corpus size is small (<10k chunks for EU AI Act + 3 papers) so risk stays low |
| R6 | LangSmith API or pricing changes break tracing | Low | Low | LangSmith is env-gated and no-op when unset; metrics + structlog give fallback observability |

---

## 12. Decision Log

| ID | Decision | Alternatives Considered | Rationale |
|---|---|---|---|
| D-001 | Modular monolith (single FastAPI service) | Microservices per domain | One request lifecycle spans all domains; no independent scaling need at this corpus size |

| | | | |
|---|---|---|---|
| D-002 | Postgres + pgvector for both BM25 (tsvector) and ANN | Qdrant/Weaviate; rank-bm25 in-memory | One service to operate; both indexes in the same transaction |
| D-003 | Reciprocal Rank Fusion (k=60) for score fusion | Weighted normalized scores; Borda count | RRF needs no per-source score normalization; k=60 is the literature default |
| D-004 | bge-small-en-v1.5 (384-dim) for embeddings | all-MiniLM-L6-v2 (faster but lower quality); bge-large (heavier) | Best small-model RAG quality at 384-dim; CPU-friendly |
| D-005 | cross-encoder/ms-marco-MiniLM-L-6-v2 for reranking | ColBERT, mxbai-rerank | Smallest production-credible reranker; matches brief; CPU-acceptable |
| D-006 | OpenRouter as the LLM gateway | Direct OpenAI; direct Anthropic | Single key, model-agnostic, lets us swap models in eval runs; matches user pattern |
| D-007 | RAGAS with `langchain-openai` judge against OpenRouter | DeepEval; Phoenix | Brief specifies RAGAS; LangSmith ecosystem alignment |
| D-008 | API key auth (X-API-Key) for v1 | JWT; OAuth | Internal/ops use; simple to rotate; JWT is cheap to migrate to later |
| D-009 | Streamlit dashboard | Next.js | Read-only ops UI; ships in hours; brief specifies Streamlit |
| D-010 | APScheduler in-process weekly drift job | Celery + Redis; Cron + container | Single-instance deployment; no extra service |
| D-011 | HNSW index (m=16, ef_construction=64) | IVFFlat | HNSW is the modern pgvector default with better recall/latency tradeoff at this scale |
| D-012 | Generated columns for `tsvector` (`Computed`) | Trigger-maintained tsvector | Postgres 12+ generated columns are simpler and fully indexable |
| D-013 | uv as package manager | poetry; pip-tools | Matches sibling projects; faster resolves |
| D-014 | structlog for logging | std logging only | JSON logs out of the box; trivial integration with request middleware |
| D-015 | Ship v0.1 fixture with 25 cases (LLM-seeded + reviewed), grow to 100 v1.0 | Block on hand-curating 100 cases up front | Unblocks Phase 5 verification while preserving the success-criteria target as a follow-up |
| D-016 | All LLM calls (generation + RAGAS judge) route through OpenRouter; request/config fields accept any OpenRouter-hosted model ID | Hardcode per-provider SDKs (Anthropic + OpenAI directly) | Single API surface and credential; RAGAS uses `langchain-openai` with `base_url=OPENROUTER_BASE_URL`; switching models is a one-string change at request or env level |

| | | | |
|---|---|---|---|
| D-017 | Default generation model: `anthropic/claude-haiku-4-5` | `claude-sonnet-4-6`, `gpt-4o-mini`, `gpt-4o` | Best speed/quality/cost for citation-grounded regulatory answers; confirmed by user 2026-04-13 |
| D-018 | Default judge model: `openai/gpt-4o-mini` (via OpenRouter) | `gpt-4o`, `claude-haiku-4-5`, `claude-sonnet-4-6` | RAGAS prompts are OpenAI-tuned; mini keeps CI cost bounded; confirmed by user 2026-04-13 |
| D-019 | API key (`X-API-Key`) is the committed v1 auth scheme | JWT Bearer, no-auth | Ops/dashboard/CI use; JWT is a cheap future migration when a public web UI is added; confirmed by user 2026-04-13 |

---

## 13. Execution Handoff

Blueprint authored by Claude Code on 2026-04-13. Ready for Codex execution.

### Open Items Before Codex Can Start

All four product decisions were resolved with the user on 2026-04-13:

- [x] Default generation model: `anthropic/claude-haiku-4-5`. Any OpenRouter-hosted model ID is accepted at runtime via `AnswerRequest.model` / `DEFAULT_GENERATION_MODEL`.
- [x] Default RAGAS judge model: `openai/gpt-4o-mini`, routed through OpenRouter. Any OpenRouter-hosted model ID is accepted via `EvalRunCreate.judge_model` / `DEFAULT_JUDGE_MODEL`.
- [x] Golden fixture strategy: ship 25-case v0.1 (LLM-seeded + manually reviewed) to unblock Phase 5; follow-up issue tracks expansion to 100-case v1.0.
- [x] Auth: API key via `X-API-Key` is the committed v1 scheme.

Remaining prerequisites:

- [ ] `ENV REQUIRED: OPENROUTER_API_KEY` available before Phase 4 ( `/answer` ) and Phase 5 (RAGAS).
- [ ] `ENV REQUIRED: API_KEYS` (real values) available before any non-local deploy.
- [ ] `ENV REQUIRED: LANGSMITH_API_KEY` optional; needed only if LangSmith tracing is enabled in Phase 6.

### How to Start Codex

Use the **Blueprint Execution Prompt** from `D:\Mehul-Projects\Claude_Planning_Charter.md`, substituting:

- `[PROJECT CODE AND NAME] = A2 DocIntel — Production RAG with RAGAS Eval, Hybrid Search & Drift Monitoring`
- `[ABSOLUTE REPO PATH] = d:\Mehul-Projects\DocIntel- RAG with RAGAS Eval, Hybrid Search & Drift Monitoring`
- `[BRANCH NAME] = main`
- `[ABSOLUTE PATH TO BLUEPRINT.md] = d:\Mehul-Projects\DocIntel- RAG with RAGAS Eval, Hybrid Search & Drift Monitoring\BLUEPRINT.md`

### Current Phase Status

- Phase 0 (Blueprint): Complete
- Next phase for Codex: **Phase 1 — Foundation**